01/02/2026 Day 6

return *this

- The `return *this` statement is used in member functions of a class to return a reference to the current object.
- This allows for method chaining, where multiple member functions can be called in a single statement.

```
#include <iostream>

using namespace std;

class Student{
    int rollNo;
    float marks;

public:


    Student(){
        cout << "Default cons" << endl;
        rollNo = 0;
        marks = 0;
    }

    Student(int rollNo, float marks) : rollNo(rollNo), marks(marks){

    }

    float getMarks() const {
        return marks;
    }

    // return *this allows for method chaining
    Student& setMarks(float marks) {
        this->marks = marks;
        return *this;
    }

    int getRollNo() const {
        return rollNo;
    }

    // return *this allows for method chaining
    Student& setRollNo(int rollNo) {
        this->rollNo = rollNo;
        return *this;
    }

    ~Student(){
```

```
        cout << "Destructor" << endl;
    }
};  //student class ends here

inline void func(){
    cout << "hello" << endl;
}

int main() {
    Student s;

    s.setRollNo(101).setMarks(40);

    cout <<s.getRollNo() << " | " << s.getMarks() << endl;
    return 0;
}
```

Destructor

- A destructor is a special member function of a class that is executed when an object of that class goes out of scope or is explicitly deleted.
- The destructor has the same name as the class but is preceded by a tilde (~) and does not take any parameters nor does it return any value.
- The primary purpose of a destructor is to release any resources that the object may have acquired during its lifetime, such as memory, file handles, or network connections.
- When an object is destroyed, the destructor is automatically called to perform cleanup tasks, ensuring that resources are properly released and preventing memory leaks or other resource-related issues.

Syntax of a destructor:

```
class ClassName {
public:
    // Constructor
    ClassName() {
        // Initialization code
    }

    // Destructor
    ~ClassName() {
        // Cleanup code
    }
};
```

```
#include <iostream>
using namespace std;
```

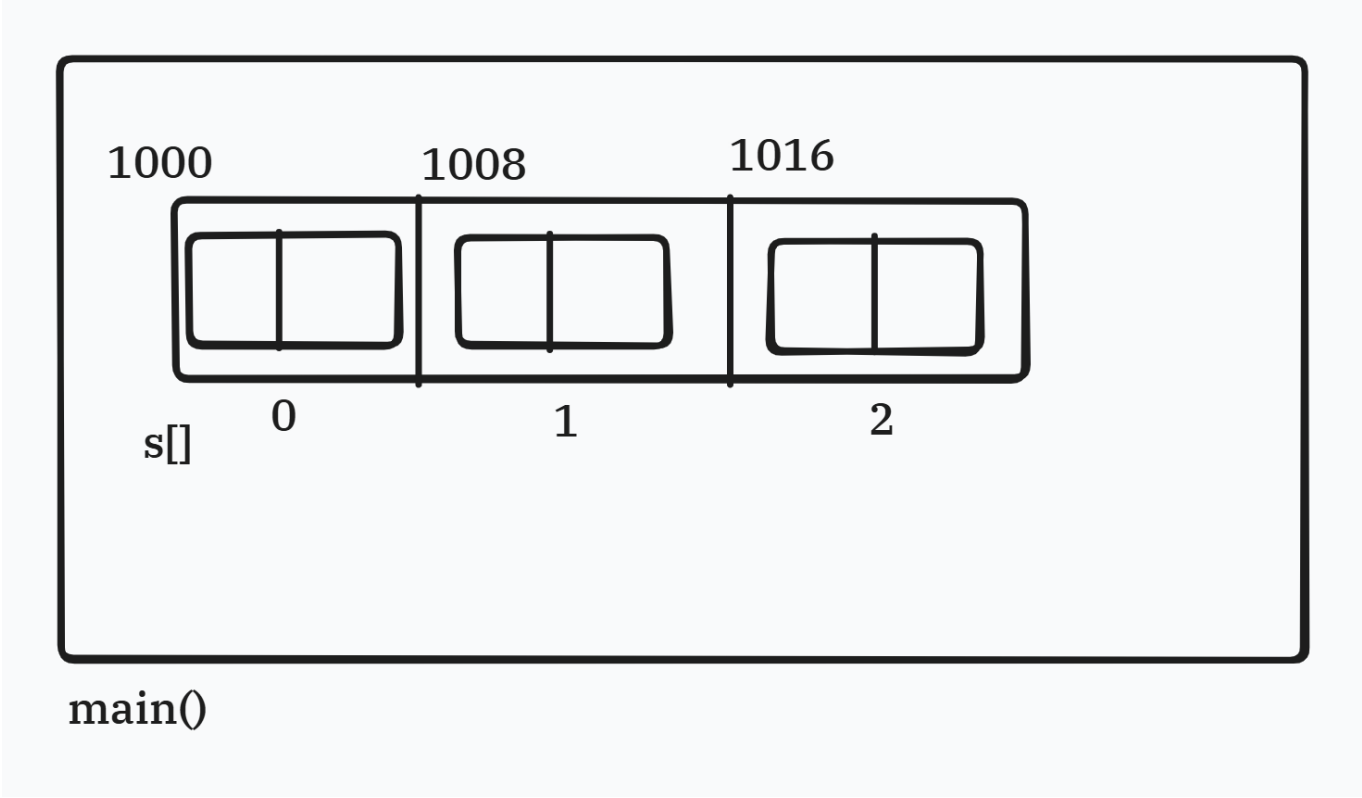
```
class Student{
    int rollNo;
    float marks;

public:
    Student(){
        cout << "Default cons" << endl;
        rollNo = 0;
        marks = 0;
    }

    Student(int rollNo, float marks, int viewCount) : rollNo(rollNo), marks(marks)
{

    }

    ~Student(){
        cout << "Destructor" << endl;
    }
};  //student class ends here

int main(){

    Student s(101,50,0);    // object create -> constructor will be called

//  Student s[3];   // array of objects -> default constructor will be called 3
times
    return 0;
}
```

Arrays of objects

- When an array of objects is created, the default constructor is called for each object in the array to initialize them.
- This means that if you have an array of objects, the default constructor will be invoked for each element in the array, ensuring that all objects are properly initialized before they are used.
- For example, if you declare an array of 3 `Student` objects, the default constructor will be called 3 times, once for each object in the array. This allows you to create and initialize multiple objects in a single statement.
- When the array of objects goes out of scope, the destructor will be called for each object in the array to clean up any resources that were allocated during their lifetime.
- This ensures that all objects in the array are properly destroyed and any resources they may have acquired are released, preventing memory leaks and other resource-related issues.

```
#include <iostream>
using namespace std;

class Student{
    int rollNo;
    float marks;
```

```
public:
    Student(){
        cout << "Default cons" << endl;
        rollNo = 0;
        marks = 0;
    }

    Student(int rollNo, float marks, int viewCount) : rollNo(rollNo), marks(marks)
{

    }

    ~Student(){
        cout << "Destructor" << endl;
    }
};  //student class ends here

int main(){

    Student s[3];   // array of objects -> default constructor will be called 3
times
    return 0;
}
```



mutable member

- A mutable member is a special type of class member that can be modified even if the containing object is declared as const.

- This allows for certain members of a class to be changed while still maintaining the overall immutability of the object.
- The mutable keyword is used to declare a member as mutable, indicating that it can be modified even in a const context.
- Member functions that are declared as const cannot modify any member variables of the class, except for those that are declared as mutable. This allows for certain members to be updated while still ensuring that the overall state of the object remains unchanged.

```
#include <iostream>
using namespace std;

class Student{
    int rollNo;
    float marks;
    mutable int viewCount;  // mutable member

public:
    Student(){
        cout << "Default cons" << endl;
        rollNo = 0;
        marks = 0;
        viewCount = 0;
    }

    Student(int rollNo, float marks, int viewCount) : rollNo(rollNo),
marks(marks), viewCount(viewCount){

    }

    void viewStudentData() const {
        viewCount++;
        cout << this->rollNo << " | " << this->marks << endl;
    }

    ~Student(){
        cout << "Destructor" << endl;
    }
};  //student class ends here

int main(){

    Student s(101,50,0);    // object create -> constructor will be called

    s.viewStudentData();    // viewCount will be updated even though
viewStudentData is a const function

    return 0;
}
```

enum (enumeration) in class

- An enum (enumeration) is a user-defined data type that consists of a set of named integral constants.
- In C++, you can define an enum inside a class to group related constants together and provide a way to represent a set of values in a more readable and maintainable way.
- The enum can be defined as a member of the class, and you can access its values using the scope resolution operator `(::)` or through an instance of the class.
- Modern C++ 11 introduced enum class (scoped enumeration) which provides better type safety and avoids name conflicts by requiring the use of the scope resolution operator to access the enumerators.

```
#include <iostream>

using namespace std;

class Student{
    int rollNo;
    float marks;
    mutable int viewCount;

public:
    // Old style enum (unscoped)
    enum Grade{A=10, B, C=30, D};

    enum Division{    // redefining error
        A, B, C, D
    };

    // New style enum (scoped) : Modern C++ 11 onwards
    enum class Grade{A=10, B, C=30, D};

    enum class Division{
        A, B, C, D
    };
//  enum ERROR_CODE{
//      NOT_FOUND = 404,
//      UNAUTHORISED = 403,
//      INTERNAL_SERVER_ERROR = 501,
//      NULL_PTR = 500
//  };


    Student(){
        cout << "Default cons" << endl;
        rollNo = 0;
        marks = 0;
    }

    Student(int rollNo, float marks, int viewCount) : rollNo(rollNo),
marks(marks), viewCount(viewCount){

    }

    float getMarks() const {
        return marks;
```

```
    }

    Student& setMarks(float marks) {
        this->marks = marks;
        return *this;
    }

    int getRollNo() const {
        return rollNo;
    }

    Student& setRollNo(int rollNo) {
        this->rollNo = rollNo;
        return *this;
    }

    void viewStudentData() const {
        viewCount++;
        cout << this->rollNo << " | " << this->marks << endl;
    }

    ~Student(){
        cout << "Destructor" << endl;
    }
};  //student class ends here

int main(){
//  s.Division::A;  // error : need to create object of class to access enum
values

s.Division d = Student::Division::A;    // accessing enum value using class name
and scope resolution operator

//    for modern enum class: static_cast is required to convert enum value to int
    cout << static_cast<int>(s.Division::A)<<endl;

    // for older enum
//  cout << s.A << endl;
//  cout << s.B << endl;
//  cout << s.C << endl;
//  cout << s.D << endl;

    return 0;
}
```

Inline functions

- An inline function is a function that is expanded in line when it is called, rather than being invoked through the usual function call mechanism.
- The `inline` keyword is used to suggest to the compiler that it should attempt to expand the function inline, which can potentially improve performance by eliminating the overhead of a function call.

- However, the decision to inline a function is ultimately up to the compiler, and it may choose not to inline a function if it determines that doing so would not be beneficial or if the function is too complex.
- Inline functions are typically used for small, simple functions that are called frequently, as the overhead of a function call can be significant in such cases. However, for larger or more complex functions, the compiler may choose to generate a regular function call instead of inlining the function.

> Note: Member functions that are defined inside the class definition are implicitly treated as inline functions by the compiler, even if the `inline` keyword is not explicitly used.

```
#include <iostream>
using namespace std;
class Student{
    int rollNo;
    float marks;
public:
    Student(){
        cout << "Default cons" << endl;
        rollNo = 0;
        marks = 0;
    }

    Student(int rollNo, float marks) : rollNo(rollNo), marks(marks){

    }
    // member function are by default inline if defined inside the class
definition
    inline float getMarks() const { // inline function
        return marks;
    }
}; //student class ends here

// global inline function
inline void func(){ // inline function
    cout << "hello" << endl;
}

int main(){
    func(); // inline function call
    return 0;
}
```

Dynamic memory (new / delete)

- Dynamic memory allocation in C++ allows you to allocate memory at runtime using the `new` operator and deallocate it using the `delete` operator.
- The `new` operator is used to allocate memory for a single object or an array of objects, and it returns a pointer to the allocated memory.

The syntax for using `new`:

```
Type* pointer = new Type; // for single object
Type* pointer = new Type[size]; // for array of objects
```

- The `delete` operator is used to deallocate memory that was previously allocated with `new`.
- It is important to use `delete` to free up memory and prevent memory leaks.

The syntax for using `delete`:

```
delete pointer; // for single object
delete[] pointer; // for array of objects
```

malloc vs new

- `malloc` is a C library function that allocates a block of memory of a specified size and returns a pointer to the beginning of the block. It does not call any constructors or initialize the memory, and it returns a void pointer that needs to be cast to the appropriate type.
- `new` is a C++ operator that allocates memory for an object or an array of objects and calls the constructor to initialize the memory. It returns a pointer of the appropriate type, so no casting is needed.
- When using `malloc`, you need to manually free the allocated memory using `free()`, while with `new`, you can use `delete` to automatically call the destructor and free the memory.

`malloc`	`new`
Allocates a block of memory of a specified size and returns a void pointer.	Allocates memory for an object or an array of objects and calls the constructor to initialize the memory.
Does not call any constructors or initialize the memory.	Calls the constructor to initialize the memory.
Returns a void pointer that needs to be cast to the appropriate type.	Returns a pointer of the appropriate type, so no casting is needed.
Memory allocated with `malloc` must be manually freed using `free()`.	Memory allocated with `new` can be freed using `delete`, which automatically calls the destructor.

```
#include <iostream>

using namespace std;

class Student{
    int rollNo;
    float marks;

public:
```

```
    Student(){
        cout << "Default Constructor..." << endl;
    }

    float getMarks() const {
        return marks;
    }

    void setMarks(float marks) {
        this->marks = marks;
    }

    int getRollNo() const {
        return rollNo;
    }

    void setRollNo(int rollNo) {
        this->rollNo = rollNo;
    }

    ~Student(){
            cout << "Desctructor..." << endl;
    }
};

int main(){

    Student s1;     // s1 object created on stack

    Student* s2 = new Student();        // s2 object created on heap

    Student* s3 = (Student*)malloc(sizeof(Student));    // create memory on heap
without calling constructor

    cout <<  "in main.."<< endl;

    free(s3);   // free the memory by malloc
    delete s2;  // calls destructor to free the memory
}
```

Dynamic arrays (1D)

- Dynamic arrays are arrays that can grow or shrink in size during runtime, allowing for more flexible memory management.
- In C++, you can create a dynamic array using the `new` operator, which allocates memory for the array on the heap.

Syntax:

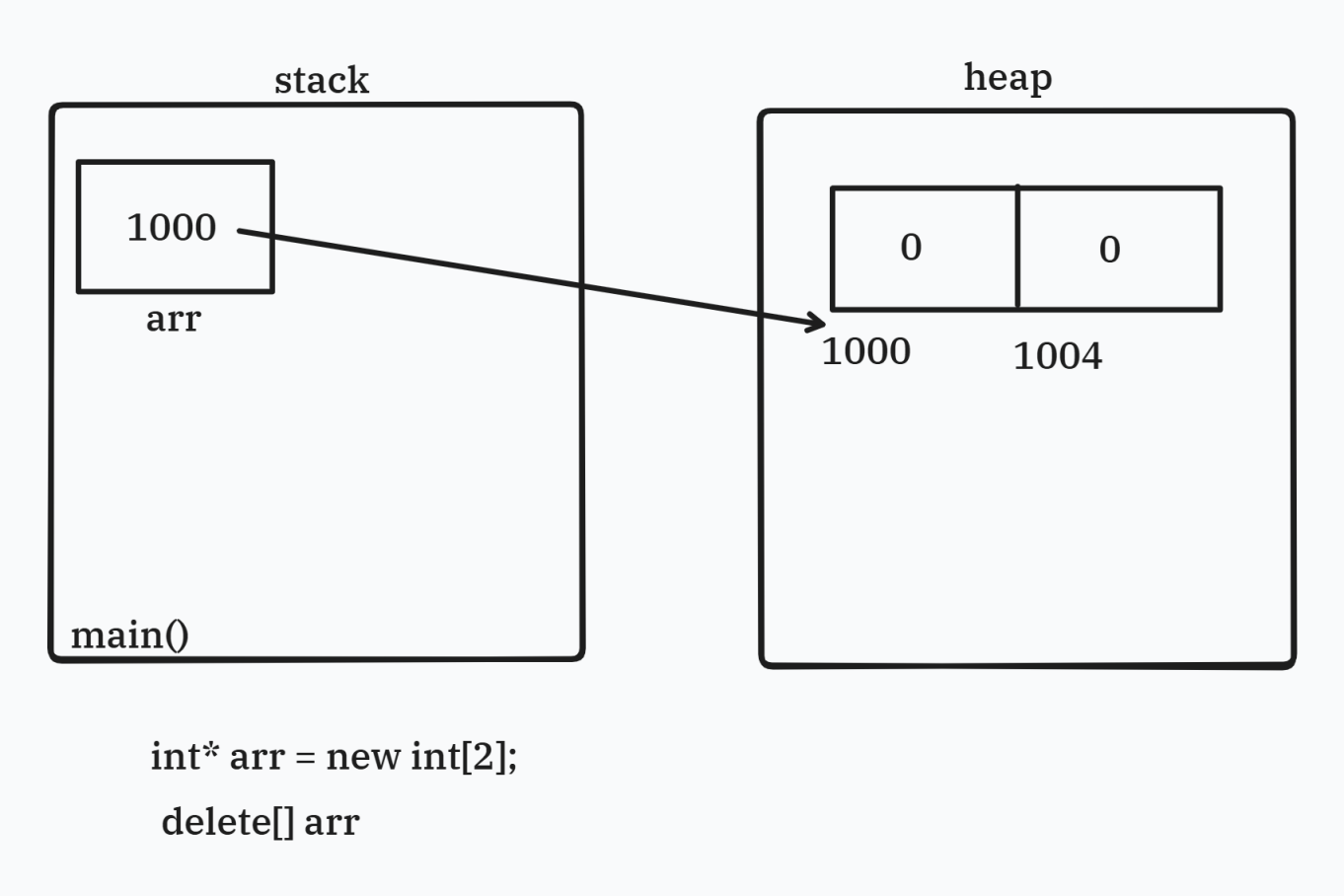
```
Type* arrayName = new Type[size];
```

- To deallocate the memory used by a dynamic array, you can use the `delete[]` operator, which calls the destructor for each element in the array and frees the memory.

Syntax:

```
delete[] arrayName;
```

```
#include <iostream>
using namespace std;

int main(){

//  int arr[3]; // creation of array on stack

    int* arr = new int[3]; // creation of array on heap

    // &arr + 1

//  delete arr; //memory leakage

    delete[] arr;   //new[] = delete[]

    return 0;
}
```



Dynamic 2D arrays

- Dynamic 2D arrays are arrays of arrays that can be created at runtime, allowing for more flexible memory management.
- In C++, you can create a dynamic 2D array using a pointer to a pointer, which allows you to allocate memory for the array on the heap.

Syntax:

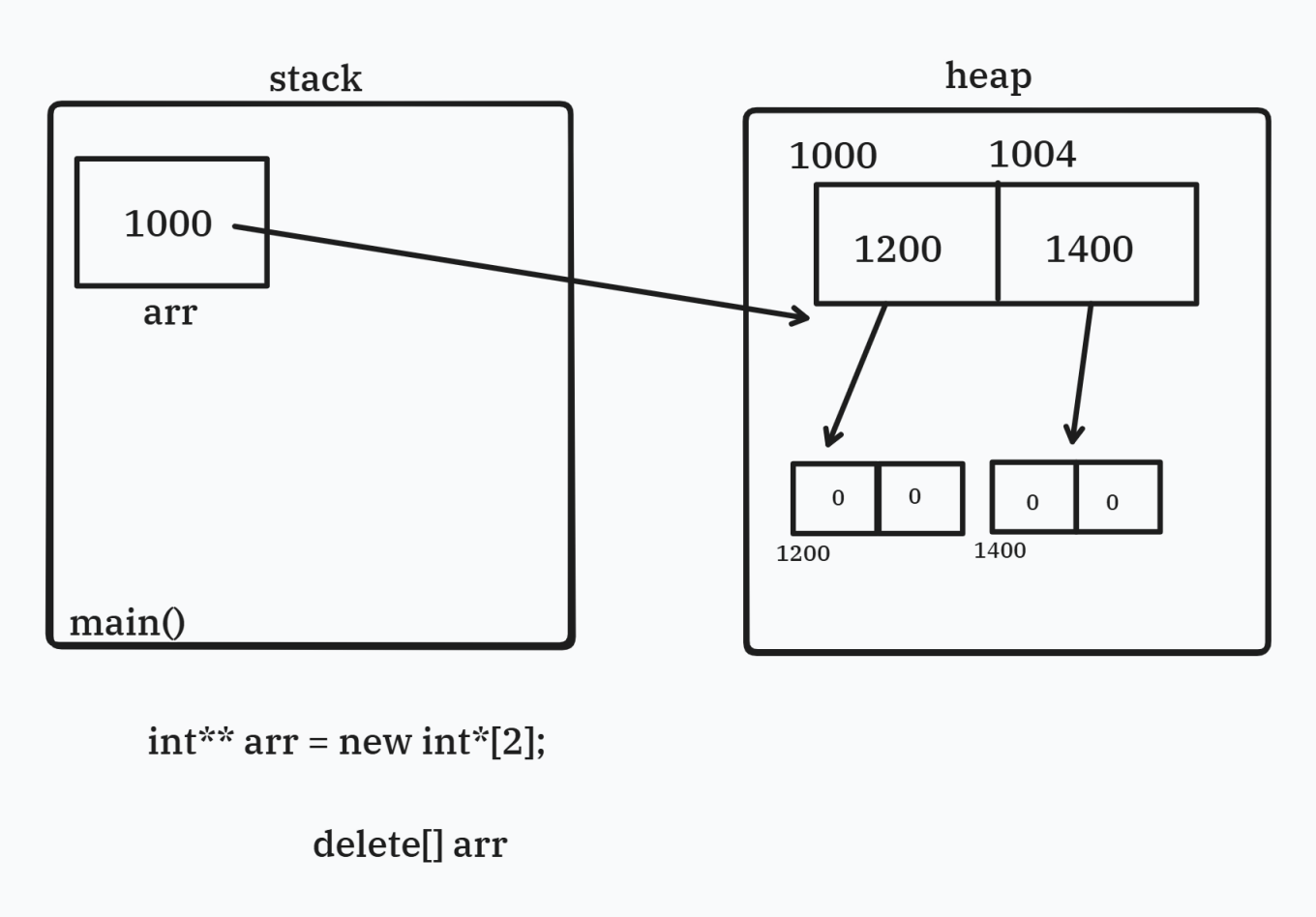
```
Type** arrayName = new Type*[rows];
for (int i = 0; i < rows; i++) {
    arrayName[i] = new Type[columns];
}
```

- To deallocate the memory used by a dynamic 2D array, you need to first delete each row of the array and then delete the array of pointers.

Syntax:

```
for (int i = 0; i < rows; i++) {
    delete[] arrayName[i]; // delete each row
}
delete[] arrayName; // delete the array of pointers
```

```
#include <iostream>
using namespace std;

int main(){
    int rows, cols;

    int** mat = new int*[rows];

    for(int i=0; i<rows; i++){
        mat[i] = new int[cols];
    }

    for(int i=0; i<rows; i++){
        delete[] mat[i];    // delete each row
    }
    delete[] mat;   //new[] = delete[]

    return 0;
}
```



Pointer arithmetic

- Pointer arithmetic refers to the operations that can be performed on pointers in C++, such as addition, subtraction, and comparison.
- When you perform pointer arithmetic, the operations are based on the size of the data type that the pointer is pointing to.
- For example, if you have a pointer to an integer, adding 1 to the pointer will actually move it to the next integer in memory, which is typically 4 bytes away.
- Pointer arithmetic can be used to traverse arrays, access elements in a data structure, or perform calculations based on memory addresses.

Array indexing vs pointer arithmetic

- Array indexing and pointer arithmetic are two ways to access elements in an array in C++.

- Array decay: When you use an array name in an expression, it decays to a pointer to the first element of the array. This means that `arr[i]` is equivalent to `*(arr + i)`, where `arr` is the name of the array and `i` is the index.

```
//Array Decay example
int arr[5] = {10, 20, 30, 40, 50};

// compiler treats arr as a pointer to the first element of the array
cout << arr << endl;
```

- arr is not a pointer variable, but it behaves like a pointer to the first element of the array.
- This is known as array decay, where the array name decays to a pointer when used in an expression.

char* is a pointer to a character, and it can be used to point to a string literal or a character array.

```
char* str = "Hello, World!"; // str points to the first character of the string
literal
```

Allowed pointer arithmetic operations:

- `+` (addition)
- `-` (subtraction)
- `++` (increment)
- `--` (decrement)
- `==` (equality comparison)

etc.

Not allowed pointer arithmetic operations:

- `*` (multiplication)
- `/` (division)
- `%` (modulus)

etc.

```
#include <iostream>
using namespace std;

// without pointer variable
int main(){
    int arr[5] = {10, 20, 30, 40, 50};

    cout << *arr << endl;   // output: 10

    cout << *(arr + 1) << endl; // output: 20

    cout << arr << endl;    // output: address of the first element of the array

    cout << arr + 1 << endl;    // output: address of the second element of the
array

    return 0;
}

// with pointer variable
int main1(){
    int arr[5] = {10, 20, 30, 40, 50};
    int* ptr = arr; // points to the first element of the array

    cout << *ptr << endl;   // output: 10

    ptr++;  // move to the next element
    cout << *ptr << endl;   // output: 20

    ptr += 2;   // move two elements ahead
    cout << *ptr << endl;   // output: 40

    return 0;
}
```

Pointer vs reference recap

```
#include <iostream>
using namespace std;
int main(){
    int a = 10;
    int* ptr = &a;  // pointer to a
    int& ref = a;   // alias to a

    return 0;
}
```

Shallow copy problem

- Shallow copy is a technique used in object-oriented programming to create a new object that is a copy of an existing object, but with shared memory for any dynamically allocated resources.
- When a shallow copy happens because the default copy constructor and assignment operator are used, the new object shares the same memory for any dynamically allocated resources as the original object. This can lead to issues such as double deletion, where both objects try to free the same memory, resulting in undefined behavior.
- In C++, you can implement a deep copy by defining a copy constructor and an assignment operator that properly allocate new memory and copy the contents of the original object to the new object.

Deep copy concept

- Deep copy is a technique used in object-oriented programming to create a new object that is a copy of an existing object, but with its own separate memory for any dynamically allocated resources.

Copy constructor

- A copy constructor is a special constructor in C++ that is used to create a new object as a copy of an existing object. It is called when an object is initialized with another object of the same class, passed by value, or returned from a function by value.
- The copy constructor takes a reference to an object of the same class as its parameter and creates a new object by copying the values of the member variables from the existing object to the new object.

Syntax of a copy constructor:

```
class ClassName {
public:
    // Copy constructor
    ClassName(const ClassName& other) {
        // Code to copy the member variables from 'other' to 'this'
    }
};

int main() {
    ClassName obj1; // Default constructor is called
    ClassName obj2 = obj1; // Copy constructor is called
    return 0;
}
```

```
#include <iostream>
using namespace std;

class Student {
    int* rollNo;
public:

    Student(){
```

```
        cout << "Defualt constructor..."<<endl;
        rollNo = nullptr;
    }

    Student(int* rollNo){
        cout << "Para constructor..."<<endl;
        this->rollNo = rollNo;
    }

    Student(const Student& s){
//      this->rollNo = s.rollNo;        // shallow copy
        cout << "DC constructor..."<<endl;
        this->rollNo = new int(*s.rollNo);  // deep copy
    }

    int* getRollNo() const {
        return rollNo;
    }

    void setRollNo(int* rollNo) {
        this->rollNo = rollNo;
    }

    ~Student(){
        cout << "Destructor called..." << this << " rollNo address: " << this-
>rollNo <<endl;
        delete rollNo;
    }
};

int main() {

//  int num = 10;
//
//  int* ptrNum = &num;     // ptr variable pointing to num address
//
//  int& ref = num;         // alias for num only

    int* p = new int(122);

    Student s1(p);      // object created on stack

    Student s2 = s1;        // this is calling CC default copy constructor

//  Student s2 = s1;        // object initialization

    Student s3;     // object creation

    s3 = s1;        // object assignment

    cout << " s1 : " << &s1 << endl;
    cout << " s2 : "<< &s2 << endl;
    cout << " s3 : "<< &s3 << endl;
    cout << s1.getRollNo() << endl;
```

```
    cout << s2.getRollNo() << endl;
    cout << s3.getRollNo() << endl;

    return 0;
}
```

Assignment operator (=)

- The assignment operator (=) is a special operator in C++ that is used to assign the values of one object to another object of the same class. It is called when an existing object is assigned the value of another existing object.

Rule of Three

- The Rule of Three is a principle in C++ programming that states that if a class defines one of the following three special member functions, it should probably explicitly define all three:

1. `Destructor`
2. `Copy Constructor`
3. `Copy Assignment Operator`

> This Rule of Three is extended to the Rule of Five in C++11 and later, which includes two additional special member functions:
>
> > 4. `Move Constructor`
> > 5. `Move Assignment Operator`

Initialization vs assignment (object context)

```
#include <iostream>
using namespace std;
int main(){

    Student s1;         // initialization
    Student s2 = s1;    // initialization using copy constructor
    Student s3;         // declaration
    s3 = s1;            // assignment

    return 0;
}
```